Live Transcription and Translation App

System architecture, build plan, and starter code structure for a low-latency Japanese/English speech translation MVP.

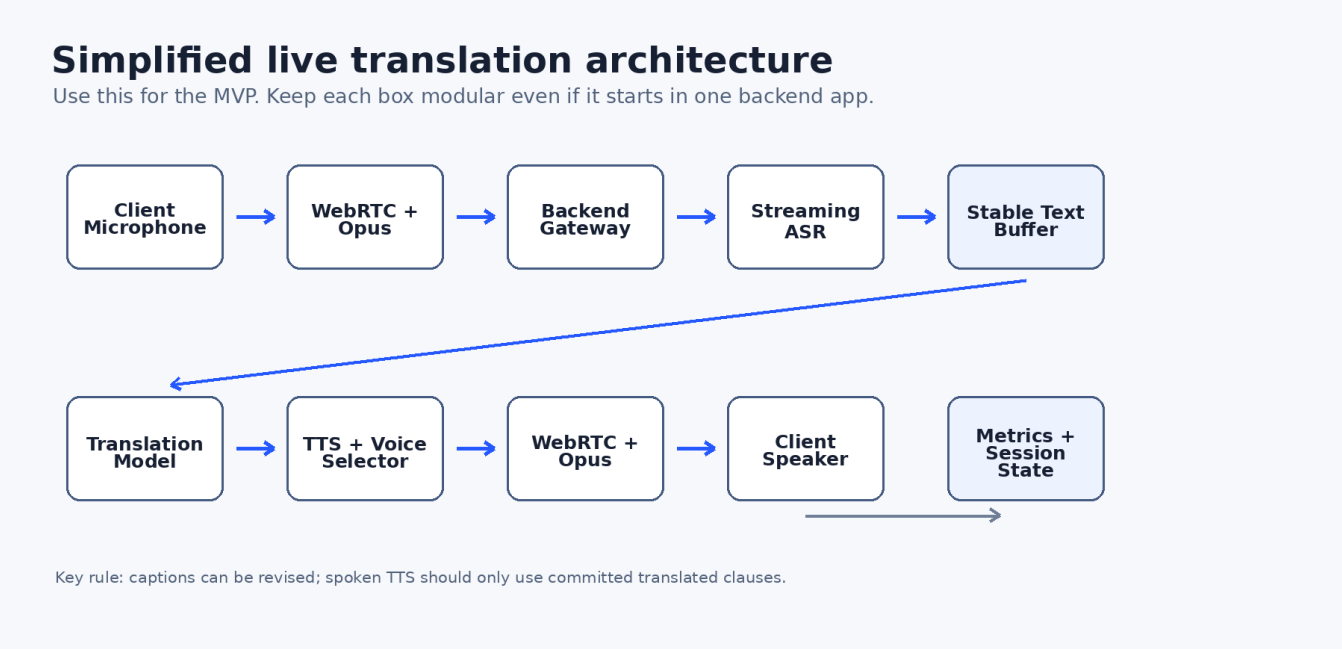
What this document gives you

- A simplified architecture for microphone audio -> ASR -> translation -> TTS -> speaker.
- A step-by-step setup approach that starts with WebRTC and fake AI modules before adding real models.
- A clean starter repository layout divided by function so each file can be edited independently.
- Starter code excerpts and a ZIP project with the full files.

Important MVP assumption: the starter project receives real microphone audio over WebRTC, but ASR, translation, and TTS are placeholders. This lets you and your intern validate the app architecture before spending time on GPU/model setup.

1. Simplified Architecture

The pipeline should be modular even if all pieces initially run inside one Python backend. The most important reliability rule is to avoid speaking text that may later be revised by the ASR model.



Runtime flow

```
User microphone
  -> WebRTC + Opus transport
  -> Python backend gateway
  -> ASR module
  -> stable text buffer
  -> translation model: Hy-MT2 / CAT-Translate / Qwen / other
  -> target-language normalization
  -> TTS + voice selector
  -> WebRTC audio or browser/server TTS playback
  -> user speaker
```

Core components

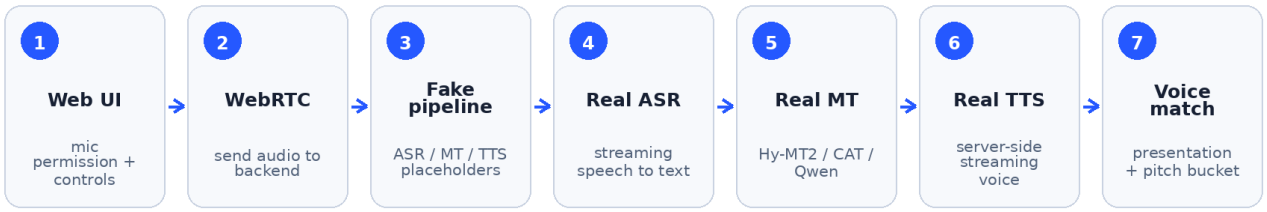
Component	Purpose	MVP implementation
Client UI	Captures microphone audio, controls session, displays source and translated text.	Static HTML/CSS/JS served by FastAPI.
WebRTC gateway	Receives browser audio and manages the peer connection.	FastAPI + aiortc /offer endpoint.
ASR module	Converts PCM audio frames to source-language text.	FakeASR now; replace with streaming ASR.
Stable text buffer	Commits only text safe enough to translate and speak.	Simple final-text commit; tune later.
Translation module	Translates committed text into the target language.	FakeTranslator now; replace with Hy-MT2/CAT/Qwen.
TTS module	Turns translated text into speech.	Browser speech synthesis placeholder now.
Voice selector	Chooses TTS voice by language and later presentation/pitch.	Neutral placeholder voice.

Boundary rule

Each module should have a small interface: input audio frames or text in, output structured events out. That keeps the app easy to test and lets you swap models without rewriting the WebRTC or UI layers.

2. Build Sequence

MVP build sequence



Do not start with heavy model integration. Verify the UI, WebRTC session, and event loop first.

Step 1 - Build the web UI

Create the microphone permission flow, language selectors, model selector, Start/Stop buttons, transcript panels, and event log.

Step 2 - Connect WebRTC

Use getUserMedia in the browser and send one audio track through RTCPeerConnection. The server accepts the SDP offer and returns an answer.

Step 3 - Keep AI fake at first

Return fake ASR and fake translation events over the data channel. This isolates transport/UI bugs from model bugs.

Step 4 - Add real ASR

Replace FakeASR with a streaming model. Output partial text, final text, timestamps, and confidence if available.

Step 5 - Add real translation

Replace FakeTranslator with Hy-MT2, CAT-Translate, LMT-60, Qwen, or another model. Keep deterministic decoding.

Step 6 - Add server-side TTS

Replace browser speechSynthesis with a TTS service that can stream audio chunks or produce short phrase audio quickly.

Step 7 - Add voice matching

Add speaker acoustic profiling and map the selected voice to neutral/masculine/feminine and low/medium/high pitch buckets.

3. Step-by-Step Local Setup

Use this sequence for the provided starter ZIP.

```
# 1) Unzip the starter project
unzip live_translator_starter.zip
cd live_translator_starter

# 2) Create a Python environment
python -m venv .venv
source .venv/bin/activate       # Windows: .venv \ Scripts \ activate

# 3) Install backend dependencies
pip install -r requirements.txt

# 4) Run the app
uvicorn server.app.main:app --reload --host 0.0.0.0 --port 8000

# 5) Open the app
http://localhost:8000

# 6) Click Start session and allow microphone access
# You should see fake Japanese/English translation events appear.
```

Expected result

- Browser asks for microphone permission.
- WebRTC connection changes from Idle -> Connecting -> Connected.
- The backend receives audio frames, groups them into short windows, and sends fake translation events.
- The UI shows source text and translation text.
- If the browser TTS checkbox is enabled, the browser speaks the translated text as a temporary placeholder.

Troubleshooting

Symptom	Likely cause	Fix
No microphone prompt	Browser permission, insecure context, or old page state.	Use localhost, refresh, check browser permissions.
/offer returns error	Backend not running or request blocked.	Confirm uvicorn is running and open /health.
Connection stuck connecting	ICE negotiation issue.	Test locally first; add TURN for remote deployments.
No speech output	Browser speechSynthesis unavailable or muted.	Check checkbox, volume, and browser support.

4. Folder Setup

This structure is divided by function so one person can work on ASR, another on translation, another on UI, and another on voice/TTS without stepping on each other.

```
live_translator_starter/
├── README.md
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── client/
│   ├── index.html
│   └── src/
│       ├── main.js
│       └── styles.css
├── server/
│   └── app/
│       ├── main.py
│       ├── config.py
│       ├── schemas.py
│       ├── webrtc_gateway.py
│       ├── pipeline/
│       │   ├── asr.py
│       │   ├── stable_buffer.py
│       │   ├── translator.py
│       │   ├── tts.py
│       │   ├── voice_selector.py
│       │   └── session.py
│       └── examples/
│           ├── hf_llm_translator.py
│           ├── latency_logger.py
│           └── voice_catalog.json
└── docs/
    └── architecture.md
```

Vibe-coding boundaries

Task	Edit these files	Do not touch first
Improve UI	client/index.html, client/src/styles.css, client/src/main.js	server/app/pipeline/*
Swap translation model	server/app/pipeline/translator.py, server/app/examples/hf_llm_translator.py	client/*
Add ASR	server/app/pipeline/asr.py, server/app/webrtc_gateway.py only if audio conversion is needed	UI layout files
Add TTS	server/app/pipeline/tts.py, client/src/main.js if playback changes	ASR and translator logic
Add voice matching	server/app/pipeline/voice_selector.py, examples/voice_catalog.json	WebRTC gateway
Add latency metrics	examples/latency_logger.py, session.py	Model wrappers

5. Starter Code: Backend Entry Point

The FastAPI app exposes /health, accepts WebRTC offers at /offer, and serves the static browser UI.

```
from pathlib import Path

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles

from server.app.config import config
from server.app.schemas import OfferRequest, OfferResponse
from server.app.webrtc_gateway import create_answer, shutdown_peer_connections


app = FastAPI(title=config.app_name)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


@app.get("/health")
async def health() -> dict[str, str]:
    return {"status": "ok"}


@app.post("/offer", response_model=OfferResponse)
async def offer(request: OfferRequest) -> OfferResponse:
    """Accept a browser WebRTC offer and return the server SDP answer."""

    answer = await create_answer(request)
    return OfferResponse(**answer)


@app.on_event("shutdown")
async def on_shutdown() -> None:
    await shutdown_peer_connections()


BASE_DIR = Path(__file__).resolve().parents[2]
CLIENT_DIR = BASE_DIR / "client"

# Keep this mount last so API routes above still work.
app.mount("/", StaticFiles(directory=CLIENT_DIR, html=True), name="client")
```

6. Starter Code: WebRTC Gateway

The gateway accepts the browser offer, receives the audio track, and emits JSON pipeline events over a WebRTC data channel.

```
import asyncio
import json
from typing import Awaitable, Callable, Optional

from aiortc import RTCPeerConnection, RTCSessionDescription
from aiortc.rtcdatachannel import RTCDataChannel
from aiortc.mediastreams import MediaStreamTrack

from server.app.config import config
from server.app.pipeline.session import TranslationSession
from server.app.schemas import OfferRequest


PEER_CONNECTIONS: set[RTCPeerConnection] = set()


async def create_answer(request: OfferRequest) -> dict[str, str]:
    """Create a WebRTC answer for a browser offer.

    The starter receives the browser audio track and emits JSON events over a
    WebRTC data channel. Server-side TTS audio can be added later by adding
    a custom MediaStreamTrack back to the peer connection.
```

```
    """

    peer_connection = RTCPeerConnection()
    PEER_CONNECTIONS.add(peer_connection)

    session = TranslationSession(
        source_language=request.source_language,
        target_language=request.target_language,
        model_name=request.model,
        voice_matching=request.voice_matching,
    )
    data_channel: Optional[RTCDataChannel] = None

    async def emit(message: dict) -> None:
        if data_channel and data_channel.readyState == "open":
            data_channel.send(json.dumps(message, ensure_ascii=False))

    @peer_connection.on("datachannel")
    def on_datachannel(channel: RTCDataChannel) -> None:
        nonlocal data_channel
        data_channel = channel

        @channel.on("message")
        def on_message(message: str) -> None:
            # Add client-side commands here later, for example:
            # - update glossary
            # - change target language
            # - interrupt TTS playback
            print(f"client message: {message}")

    @peer_connection.on("track")
    def on_track(track: MediaStreamTrack) -> None:
        if track.kind == "audio":
            asyncio.create_task(consume_audio_track(track, session, emit))

    @peer_connection.on("connectionstatechange")
    async def on_connectionstatechange() -> None:
        if peer_connection.connectionState in {"failed", "closed", "disconnected"}:
            await peer_connection.close()
            PEER_CONNECTIONS.discard(peer_connection)

    offer = RTCSessionDescription(sdp=request.sdp, type=request.type)
    await peer_connection.setRemoteDescription(offer)
    answer = await peer_connection.createAnswer()
    await peer_connection.setLocalDescription(answer)

    return {
        "sdp": peer_connection.localDescription.sdp,
        "type": peer_connection.localDescription.type,
    }


async def consume_audio_track(
    track: MediaStreamTrack,
    session: TranslationSession,
    emit: Callable[[dict], Awaitable[None]],
) -> None:
    """Collect WebRTC audio frames and pass windows into the AI pipeline.

    aiortc decodes the browser's Opus audio and exposes PCM-like AudioFrame
    objects. The fake pipeline ignores the actual audio, but this function is
    where a real ASR implementation would resample and feed frames to the
    model.
    """

    frames = []
    accumulated_seconds = 0.0

    while True:
        try:
            frame = await track.recv()
        except Exception as exc:
            await emit({"type": "track_closed", "detail": str(exc)})
            return

        frames.append(frame)
        sample_rate = getattr(frame, "sample_rate", 48000) or 48000
        samples = getattr(frame, "samples", 0) or 0
# ... see ZIP for full file
```

7. Starter Code: Session Pipeline

The session object wires ASR, stable text commit, translation, and voice selection together. This is the clean seam for replacing fake modules with real ones.

```
from dataclasses import dataclass, field
from typing import Iterable, Optional
from uuid import uuid4

from server.app.pipeline.asr import FakeASR
from server.app.pipeline.stable_buffer import StableTextBuffer
from server.app.pipeline.translator import FakeTranslator
from server.app.pipeline.voice_selector import VoiceSelector


@dataclass
class TranslationSession:
    source_language: str
    target_language: str
    model_name: str
    voice_matching: bool = False
    session_id: str = field(default_factory=lambda: str(uuid4()))

    def __post_init__(self) -> None:
        self.asr = FakeASR()
        self.translator = FakeTranslator(model_name=self.model_name)
        self.stable_buffer = StableTextBuffer()
        self.voice_selector = VoiceSelector()

    async def process_audio_window(self, frames: Iterable[object]) -> Optional[dict]:
        asr_result = await self.asr.transcribe_pcm(frames, self.source_language)
        committed_text = self.stable_buffer.commit(asr_result)

        if not committed_text:
            return {
                "type": "asr_partial",
                "session_id": self.session_id,
                "source": {
                    "text": asr_result.text,
                    "language": asr_result.language,
                    "confidence": asr_result.confidence,
                },
            }

        translated_text = await self.translator.translate(
            committed_text,
            source_language=self.source_language,
            target_language=self.target_language,
        )
        voice = self.voice_selector.select(self.target_language, self.voice_matching)

        return {
            "type": "translation",
            "session_id": self.session_id,
            "source": {
                "text": committed_text,
                "language": self.source_language,
                "confidence": asr_result.confidence,
            },
            "translation": {
                "text": translated_text,
                "language": self.target_language,
                "model": self.model_name,
            },
            "voice": {
                "voice_id": voice.voice_id,
                "presentation": voice.presentation,
                "pitch_bucket": voice.pitch_bucket,
            },
        }
```

8. Starter Code: Fake ASR and Translator

These placeholders let you test the live app while model selection is still in progress.

server/app/pipeline/asr.py

```
from dataclasses import dataclass, field
from itertools import cycle
from typing import Iterable, Protocol


@dataclass
class ASRResult:
    text: str
    is_final: bool
    confidence: float
    language: str


class ASR(Protocol):
    async def transcribe_pcm(self, frames: Iterable[object], source_language: str) ->
        ASRResult:
        ...


@dataclass
class FakeASR:
    """Fake ASR used to test the app before adding a real speech model."""

    _ja_samples: object = field(default_factory=lambda: cycle([
        "えっと、明日の会議を午後三時に変更できますか？",
        "すみません、もう一回ゆっくり言ってもらえますか？",
        "この資料は今日中に確認します。",
    ]))
    _en_samples: object = field(default_factory=lambda: cycle([
        "Actually, can we move the meeting to next Wednesday instead?",
        "Could you please say that one more time slowly?",
        "I will review this document by the end of today.",
    ]))

    async def transcribe_pcm(self, frames: Iterable[object], source_language: str) ->
        ASRResult:
        text = next(self._ja_samples if source_language == "ja" else self._en_samples)
        return ASRResult(
            text=text,
            is_final=True,
            confidence=0.99,
            language=source_language,
        )
```

server/app/pipeline/translator.py

```
from dataclasses import dataclass
from typing import Protocol


class Translator(Protocol):
    async def translate(self, text: str, source_language: str, target_language: str) -> str:
        ...


@dataclass
class FakeTranslator:
    """Fake translator used while the audio/WebRTC plumbing is being built."""

    model_name: str = "fake"

    async def translate(self, text: str, source_language: str, target_language: str) -> str:
        ja_to_en = {
            "えっと、明日の会議を午後三時に変更できますか？": "Um, can we move tomorrow's meeting to 3 p.m.?",
            "すみません、もう一回ゆっくり言ってもらえますか？": "Sorry, could you say that one more time slowly?",
            "この資料は今日中に確認します。": "I will review this document by the end of today.",
        }
        en_to_ja = {
            "Actually, can we move the meeting to next Wednesday instead?":
                "やっぱり、会議を来週の水曜日に変更できますか？",
            "Could you please say that one more time slowly?": "もう一度ゆっくり言っていただけますか？",
            "I will review this document by the end of today.": "この資料は今日中に確認します。",
        }
        if source_language == "ja" and target_language == "en":
            return ja_to_en.get(text, f"[fake EN translation] {text}")
        if source_language == "en" and target_language == "ja":
            return en_to_ja.get(text, f"[fake JA translation] {text}")
        return text
```

9. Starter Code: Client UI

The client captures microphone audio, creates a peer connection, sends the SDP offer to the server, and renders translation events.

client/index.html excerpt

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Live Translator Starter</title>
    <link rel="stylesheet" href="/src/styles.css" />
  </head>
  <body>
    <main class="shell">
      <section class="hero">
        <div>
          <p class="eyebrow">Real-time speech translation starter</p>
          <h1>Live Translator</h1>
          <p class="subtitle">Mic → WebRTC → ASR → translation → TTS. The first build uses
            fake AI modules so you can wire up the app safely.</p>
        </div>
        <div class="status-card">
          <span id="connectionDot" class="dot"></span>
          <span id="connectionStatus">Idle</span>
        </div>
      </section>

      <section class="panel controls">
        <label>
          Source language
          <select id="sourceLanguage">
            <option value="ja" selected>Japanese</option>
            <option value="en">English</option>
          </select>
        </label>

        <label>
          Target language
          <select id="targetLanguage">
            <option value="en" selected>English</option>
            <option value="ja">Japanese</option>
          </select>
        </label>

        <label>
          Translation model
          <select id="modelName">
            <option value="fake" selected>Fake translator</option>
            <option value="tencent/Hy-MT2-1.8B">Hy-MT2-1.8B</option>
            <option value="cyberagent/CAT-Translate-3.3b">CAT-Translate-3.3B</option>
            <option value="Qwen/Qwen3-8B">Qwen3-8B</option>
          </select>
        </label>

        <label class="check-row">
          <input id="browserTts" type="checkbox" checked />
          Use browser TTS placeholder
        </label>

        <label class="check-row">
          <input id="voiceMatching" type="checkbox" />
          Enable voice matching placeholder
        </label>

        <div class="button-row">
          <button id="startButton" class="primary">Start session</button>
          <button id="stopButton" disabled>Stop</button>
        </div>
      </section>

      <section class="grid">
        <article class="panel transcript-card">
          <div class="panel-title">
            <h2>Source transcript</h2>
            <span class="pill">ASR</span>
# ... see ZIP for full file
```

client/src/main.js excerpt

```
let peerConnection = null;
let localStream = null;
let dataChannel = null;

const els = {
  startButton: document.querySelector('#startButton'),
  stopButton: document.querySelector('#stopButton'),
  clearLogButton: document.querySelector('#clearLogButton'),
  connectionDot: document.querySelector('#connectionDot'),
  connectionStatus: document.querySelector('#connectionStatus'),
  sourceLanguage: document.querySelector('#sourceLanguage'),
  targetLanguage: document.querySelector('#targetLanguage'),
  modelName: document.querySelector('#modelName'),
  browserTts: document.querySelector('#browserTts'),
  voiceMatching: document.querySelector('#voiceMatching'),
  sourceTranscript: document.querySelector('#sourceTranscript'),
  translationTranscript: document.querySelector('#translationTranscript'),
  eventLog: document.querySelector('#eventLog'),
};

els.startButton.addEventListener('click', startSession);
els.stopButton.addEventListener('click', stopSession);
els.clearLogButton.addEventListener('click', () => { els.eventLog.textContent = ''; });

async function startSession() {
  setStatus('connecting', 'Connecting');
  els.startButton.disabled = true;
  logEvent('Requesting microphone permission...');

  try {
    localStream = await navigator.mediaDevices.getUserMedia({
      audio: {
        echoCancellation: true,
        noiseSuppression: true,
        autoGainControl: true,
      },
      video: false,
    });

    peerConnection = new RTCPeerConnection({
      iceServers: [{ urls: 'stun:stun.l.google.com:19302' }],
    });

    dataChannel = peerConnection.createDataChannel('events');
    dataChannel.onopen = () => logEvent('Data channel opened.');
    dataChannel.onmessage = (event) => handleServerEvent(event.data);
    dataChannel.onclose = () => logEvent('Data channel closed.');

    peerConnection.onconnectionstatechange = () => {
      const state = peerConnection.connectionState;
      logEvent(`Peer connection state: ${state}`);
      if (state === 'connected') setStatus('connected', 'Connected');
      if (state === 'failed' || state === 'disconnected') setStatus('failed', state);
    };

    for (const track of localStream.getAudioTracks()) {
      peerConnection.addTrack(track, localStream);
    }

    const offer = await peerConnection.createOffer();
# ... see ZIP for full file
```

10. Swapping in a Real Translation Model

Start with a local wrapper around a chat-style or translation-specific model. Keep the application code calling a simple translate(text, source_language, target_language) method so you can swap models without changing the whole app.

```
"""Optional Transformers translator wrapper.

This is not imported by the starter server. Copy the parts you need into
`server/app/pipeline/translator.py` when you are ready to test a local model.

Install optional dependencies first:

    pip install torch transformers accelerate sentencepiece

Example model IDs to try:

    tencent/Hy-MT2-1.8B
    cyberagent/CAT-Translate-3.3b
    Qwen/Qwen3-8B
"""

from __future__ import annotations

import asyncio
from dataclasses import dataclass


@dataclass
class HFChatTranslator:
    model_id: str
    max_new_tokens: int = 160

    def __post_init__(self) -> None:
        import torch
        from transformers import AutoModelForCausalLM, AutoTokenizer

        self.torch = torch
        self.tokenizer = AutoTokenizer.from_pretrained(self.model_id,
            trust_remote_code=True)
        self.model = AutoModelForCausalLM.from_pretrained(
            self.model_id,
            torch_dtype=torch.bfloat16,
            device_map="auto",
            trust_remote_code=True,
        )

    async def translate(self, text: str, source_language: str, target_language: str) -> str:
        return await asyncio.to_thread(self._translate_sync, text, source_language,
            target_language)

    def _translate_sync(self, text: str, source_language: str, target_language: str) -> str:
        prompt = (
# ... see ZIP for full file
```

Suggested model test order for Japanese/English

Priority	Model	Why test it
1	CAT-Translate-3.3B	Japanese/English-specific and small enough for fast testing.
2	Hy-MT2-1.8B	Fast multilingual MT baseline.
3	CAT-Translate-7B	Japanese/English quality fallback.
4	LMT-60-8B	Qwen-based multilingual MT model.
5	Qwen3-8B or Qwen3-14B	General LLM fallback for messy ASR and context handling.

11. What to Replace Next

Replace one module at a time. Do not add ASR, real translation, server TTS, and voice matching in the same sprint, because debugging the source of latency or errors will become much harder.

Sprint	Goal	Acceptance test
1	WebRTC + fake pipeline	Mic permission works; fake translations arrive in UI every 1-2 seconds.
2	Real ASR	Server receives real transcript text from speech. Translation can still be fake.
3	Real translation	Committed ASR text is translated by selected model. TTS can still be browser placeholder.
4	Server TTS	Translated audio is produced by a TTS module and played without long gaps.
5	Voice selector	User/manual voice preference works; acoustic pitch bucket can be added later.
6	Latency metrics	Every segment logs ASR, MT, TTS, and end-to-end timing.

Latency targets to measure

```
audio_received_at
asr_partial_at
source_committed_at
translation_started_at
translation_finished_at
tts_first_audio_at
audio_played_at
```

12. Voice Matching Plan

Do not infer a person's gender identity from voice. Treat this as voice presentation matching and provide manual override options. Start with neutral voices, then add masculine/feminine/neutral preferences and low/medium/high pitch buckets.

voice_catalog.json

```
{
  "voice_id": "en-masculine-low-01",
  "language": "en",
  "presentation": "masculine",
  "pitch_bucket": "low",
  "median_pitch_reference": 110,
  "supports_streaming": true
}
```

Selection logic

- Filter voices by target language.
- Apply user-selected presentation: neutral, masculine, feminine, or no preference.
- Estimate the speaker pitch percentile from voiced frames rather than hard-coding one Hz threshold.
- Map percentile to low/medium/high and select the closest voice in the catalog.
- Lock the chosen voice per speaker for the session so it does not jump between sentences.

13. References Used for the Starter Choices

These sources are included so an intern can look up the underlying APIs and model docs.

[1]	MDN getUserMedia: browser microphone permission and MediaStream capture.	https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia
[2]	MDN RTCPeerConnection: browser WebRTC peer connection API.	https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection
[3]	aiortc API docs: Python RTCPeerConnection and offer/answer handling.	https://aiortc.readthedocs.io/en/latest/api.html
[4]	FastAPI StaticFiles docs: serving the static client folder.	https://fastapi.tiangolo.com/tutorial/static-files/
[5]	Hugging Face Transformers chat templates: model chat prompt formatting.	https://huggingface.co/docs/transformers/en/chat_templating
[6]	Hy-MT2 paper/model info: multilingual MT model family.	https://arxiv.org/html/2605.22064v1
[7]	CAT-Translate model card: Japanese/English bidirectional translation models.	https://huggingface.co/cyberagent/CAT-Translate-3.3b
[8]	Qwen3-8B model card: general multilingual LLM baseline.	https://huggingface.co/Qwen/Qwen3-8B

Deliverables

Use the companion ZIP file for the full starter project. The PDF includes excerpts, but the ZIP is the canonical code artifact.